



INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

19 April 2024

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 2040

SYSTEM PROGRAMMING

WEEK 10 Lecture 2

LINKING

PART 3



SOC 2040 SYSTEM PROGRAMMING

CASE STUDY LIBRARY INTERPOSITIONING



Case Study: Library Interpositioning

- ★ **Library interpositioning** : powerful linking technique that allows programmers to intercept calls to shared library functions and execute their own code instead
- ★ **Interpositioning can occur at:**
 - **Compile time**: When the source code is compiled
 - **Link time**: When the relocatable object files are statically linked to form an executable object file
 - **Load/run time**: When an executable object file is loaded into memory, dynamically linked, and then executed.

Some Interpositioning Applications

★ Security

- Confinement (sandboxing)
- Behind the scenes encryption

★ Debugging

- In 2014, two Facebook engineers debugged a treacherous 1-year old bug in their iPhone app using interpositioning
- Code in the SPDY networking stack was writing to the wrong location
- Solved by intercepting calls to Posix write functions (write, writev, pwrite)

Source: Facebook engineering blog post at

<https://code.facebook.com/posts/313033472212144/debugging-file-corruption-on-ios/>

Some Interpositioning Applications

★ Monitoring and Profiling

- Using inter-positioning, one could trace the number of times a particular library function is called, validate and trace its input and output values, or even replace it with a complete different implementations.
- Characterize call sites and arguments to functions
- Malloc tracing
 - ★ Detecting memory leaks
 - ★ **Generating address traces**

Library Interpositioning – Basic Idea

Basic Idea :

- ★ Given some target function to be interposed on, Create a wrapper function whose prototype is identical to the target function.
- ★ Using some particular interpositioning mechanism, then trick the system into calling the wrapper function instead of the target function.
- ★ The wrapper function typically executes its own logic, then calls the target function and passes its return value back to the caller.
- ★ Interpositioning can occur at compile time, link time, or run time as the program being loaded and executed.

Example program

int.c

```
#include <stdio.h>
#include <malloc.h>

int main()
{
    int *p = malloc(32);
    free(p);
    return(0);
}
```

- ★ Goal: trace the addresses and sizes of the allocated and freed blocks, without breaking the program, and without modifying the source code.
- ★ Three solutions: interpose on the `lib malloc` and `free` functions at compile time, link time, and load/run time.

Compile-time Interpositioning

```
#include <stdio.h>
#include <malloc.h>
```

Int.c

```
int main()
{
    int *p = malloc(32);
    free(p);
    return(0);
}
```

Each wrapper function in mymalloc.c calls the target function, prints a trace, and returns

```
#define malloc(size) mymalloc(size)
#define free(ptr) myfree(ptr)
```

```
void *mymalloc(size_t size);
void myfree(void *ptr);
```

malloc.h

The local malloc.h header file instructs the pre-processor to replace each call to a target function with a call to its wrapper

mymalloc.c

```
#ifndef COMPILETIME
#include <stdio.h>
#include <malloc.h>

/* malloc wrapper function */
void *mymalloc(size_t size)
{
    void *ptr = malloc(size);
    printf("malloc(%d)=%p\n",
        (int)size, ptr);
    return ptr;
}

/* free wrapper function */
void myfree(void *ptr)
{
    free(ptr);
    printf("free(%p)\n", ptr);
}
#endif
```

Compile-time Interpositioning

To compile and link the program:

```
$gcc -Wall -DCOMPILETIME -c mymalloc.c
```

```
$gcc -Wall -I. -o intc int.c mymalloc.o
```

- * Interpositioning happens because of the `-I.` argument, which tells the C preprocessor to look for `malloc.h` in the current directory before looking in the usual system directories.
- * Notice that the wrappers in `mymalloc.c` are compiled with the standard `malloc.h` header file.

Running the program gives the following trace:

```
$/intc
```

```
malloc(32)=0x1edc010
```

```
free(0x1edc010)
```

```
$
```

Link-time Interpositioning

- The Linux static Linker supports link-time interpositioning with the `--wrap` flag.
- This flag tells the linker to resolve references to symbol `f` as `__wrap_f` and to resolve references to symbol `__real_f` as `f`.

```
#include <stdio.h>
#include <malloc.h>

int main()
{
    int *p = malloc(32);
    free(p);
    return(0);
}
```

Int.c

Link-time Interpositioning

```
#ifdef LINKTIME
#include <stdio.h>

void *__real_malloc(size_t size);
void __real_free(void *ptr);

/* malloc wrapper function */
void *__wrap_malloc(size_t size)
{
    void *ptr = __real_malloc(size); /* Call libc malloc */
    printf("malloc(%d) = %p\n", (int)size, ptr);
    return ptr;
}

/* free wrapper function */
void __wrap_free(void *ptr)
{
    __real_free(ptr); /* Call libc free */
    printf("free(%p)\n", ptr);
}
#endif
```

mymalloc.c

Link-time Interpositioning

To compile the source files into relocatable object files :

```
$gcc -Wall -DLINKTIME -c mymalloc.c
```

```
$gcc -Wall -c int.c
```

To link the object files into an executable :

```
$gcc -Wall -Wl,--wrap,malloc -Wl,--wrap,free -o intl  
int.o mymalloc.o
```

The “-Wl” flag passes argument to linker, replacing each comma with a space.

*The “--wrap,malloc” arg instructs linker to resolve references in a special way:

- Refs to malloc should be resolved as __wrap_malloc
- Refs to __real_malloc should be resolved as malloc

Running the program gives the following trace :

```
$/intl
```

```
malloc(32) = 0x1aa0010
```

```
free(0x1aa0010)
```

```
$
```

Run-time Interpositioning

- Compile time interpositioning requires access to a program's source files.
- Link-time inter-positioning requires access to its relocatable object files
- However, there is a mechanism for inter-positioning at run time that requires access only to the executable object file.
- This fascinating mechanism is based on the dynamic linker's LD_PRELOAD environment variable.
- If the LD_PRELOAD environment variable is set to a list of shared library pathnames (separated by spaces or colons), then when we load and execute a program, the dynamic linker (LD-Linux.so) will search the LD_PRELOAD libraries first, before any other shared libraries, when it resolves undefined references.
- With this mechanism, we can interpose on any function in any shared library, including libc.so, when we load and execute any executable.

Load/Run-time Interpositioning

Int.c

mymalloc.c

```
#ifdef RUNTIME
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <dlfcn.h>

/* malloc wrapper function */
void *malloc(size_t size)
{
    void *(*mallocp)(size_t size);
    char *error;

    mallocp = dlsym(RTLD_NEXT, "malloc"); /* Get addr of libc malloc */
    if ((error = dlerror()) != NULL) {
        fputs(error, stderr);
        exit(1);
    }
    char *ptr = mallocp(size); /* Call libc malloc */
    printf("malloc(%d) = %p\n", (int)size, ptr);
    return ptr;
}
```

```
#include <stdio.h>
#include <malloc.h>

int main()
{
    int *p = malloc(32);
    free(p);
    return(0);
}
```

- In each wrapper, the call to dlsym returns the pointer to the target libc function.
- The wrapper then calls the target function, prints a trace and returns

Load/Run-time Interpositioning

```
/* free wrapper function */
void free(void *ptr)
{
    void (*freep)(void *) = NULL;
    char *error;

    if (!ptr)
        return;

    freep = dlsym(RTLD_NEXT, "free"); /* Get address of libc free */
    if ((error = dlerror()) != NULL) {
        fputs(error, stderr);
        exit(1);
    }
    freep(ptr); /* Call libc free */
    printf("free(%p)\n", ptr);
}
#endif
```

mymalloc.c

Load/Run-time Interpositioning

To build the shared library that contains the wrapper functions :

```
$gcc -Wall -DRUNTIME -shared -fpic -o mymalloc.so  
mymalloc.c -ldl
```

To compile the main program int.c :

```
$gcc -Wall -o intr int.c
```

To run the program intr

```
$LD_PRELOAD="./mymalloc.so"  
./intr
```

```
malloc(32) = 0xe60010
```

```
free(0xe60010)
```

```
$
```

The `LD_PRELOAD` environment variable tells the dynamic linker to resolve unresolved refs (e.g., to `malloc`) by looking in `mymalloc.so` first.

Interpositioning EXAMPLE

* DETECT & ELIMINATE MEMORY LEAKS

Main program

Declare **global counter** to count number of malloc calls **mcount=0**

Declare **global array of pointers** to hold **address/pointer** returned by every malloc call, **marray**

Write a function to **create a dynamic linked list with nodes** having student info. **Create a large number of nodes using malloc & but free only few nodes**

Insert a **fixup routine** to **detect memory leaks** and **take action to free all unallocated blocks/nodes**

L
I
B
R
A
R
Y

I
N
T
E
R
P
O
S
I
T
I
O
N
I
N
G

mymalloc wrapper function

p ← **malloc(s)**

Increment the global counter **mcount**

Add the address **p** to the global array **marray**

Print **p, s, mcount**

myfree wrapper function

free(p)

decrement the global counter **mcount**

Remove the address **p** from the global array **marray**, make that entry null

Print **p, mcount**

Interpositioning Recap

- ★ Compile Time
 - Apparent calls to malloc/free get macro-expanded into calls to mymalloc/myfree
- ★ Link Time
 - Use linker trick to have special name resolutions
malloc → __wrap_malloc
__real_malloc → malloc
- ★ Load/Run Time
 - Implement custom version of malloc/free that use dynamic linking to load library malloc/free under different names

Tools for Manipulating Object Files

- ✳ There are a number of tools available on Linux systems to understand and manipulate object files.
- ✳ In General, binutils package is especially helpful and runs on every Linux platform
- ✳ **AR** – creates static libraries, and inserts, deletes, lists and extracts members
- ✳ **STRINGS** – Lists all of the printable strings contained in an object file
- ✳ **STRIP** - Deletes symbol table information from an object file
- ✳ **NM** – Lists the symbols defined in the symbol table of an object file
- ✳ **SIZE** – Lists the names and sizes of the sections in an object file
- ✳ **READELF** – Displays the complete structure of an object file, including all of the information encoded in the ELF header. Subsumes the functionality of SIZE and NM.
- ✳ **OBJDUMP** – The mother of all binary tools. Can display all of the information in an object file. Its most useful function is disassembling the binary instructions in the .text section
- ✳ Linux systems also provide the LDD program for manipulating shared libraries
- ✳ **LDD**: Lists the shared libraries that an executable needs at run time.

SOC 2040 SYSTEM PROGRAMMING

Reading Assignments

Chapter 9 of Text Book

➤ Computer Systems : A Programmer's Perspective

- Randal E. Bryant and David R. O'Hallaron
3rd Edition, Global Edition Prentice Hall 2016
ISBN 10:1-292-10176-8**

SOC 2040
SYSTEM PROGRAMMING

END OF
WEEK 10 Lecture 2
LINKING PART 3

WISH YOU ALL THE BEST



INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

19 April 2024

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG